

Mini Store Manager — Fichier `src/app.py`

Auteur : Ezekiel Houab **Date : 12 septembre 2026** **Projet : Application Flask — Gestion de magasin**

1. Introduction

Ce document présente la version finale et fonctionnelle du fichier `src/app.py` utilisé dans l'application Flask *Mini Store Manager*. Ce fichier :

- initialise l'application Flask
- configure la clé secrète pour les sessions
- enregistre les blueprints
- génère le tableau de bord (dashboard)
- lance le serveur en mode développement

Ce document est prêt à être exporté en PDF.

2. Code complet — `src/app.py`

```
python
from flask import Flask, render_template
from customers.routes import customers_bp
from products.routes import products_bp
from orders.routes import orders_bp
from categories.routes import categories_bp
from analytics.routes import analytics_bp

from database import (
    get_all_customers,
    get_all_products,
    get_all_orders
)

from collections import defaultdict
from datetime import datetime

app = Flask(__name__)

# =====
# SECRET KEY (obligatoire pour flash(), session, etc.)
# =====
app.secret_key = "ezekiel_super_secret_key_2026"

# =====
# Enregistrement des blueprints
# =====

app.register_blueprint(customers_bp)
```

```

app.register_blueprint(products_bp)
app.register_blueprint(orders_bp)
app.register_blueprint(categories_bp)
app.register_blueprint(analytics_bp)

# =====
# Dashboard
# =====

@app.route("/")
@app.route("/dashboard")
def dashboard():
    customers = get_all_customers()
    products = get_all_products()
    orders = get_all_orders()

    # KPIs
    total_customers = len(customers)
    total_products = len(products)
    total_orders = len(orders)
    total_revenue = sum(o["total"] for o in orders)

    # Ventes par produit
    sales_by_product = defaultdict(float)
    qty_by_product = defaultdict(int)

    for o in orders:
        sales_by_product[o["product_name"]] += o["total"]
        qty_by_product[o["product_name"]] += o["quantity"]

    sales_labels = list(sales_by_product.keys())
    sales_values = list(sales_by_product.values())

    # Top 5 produits
    top_products = sorted(
        sales_by_product.items(),
        key=lambda x: x[1],
        reverse=True
    )[:5]

    top_products_labels = [name for name, total in top_products]
    top_products_values = [total for name, total in top_products]

    # Produit le plus vendu (quantité)
    if qty_by_product:
        best_product_name, best_product_qty = max(
            qty_by_product.items(),
            key=lambda x: x[1]
        )
    else:
        best_product_name, best_product_qty = None, 0

    # Stock par produit
    stock_labels = [p["name"] for p in products]
    stock_values = [p["stock"] for p in products]

    # Ventes mensuelles
    sales_by_month = defaultdict(float)
    for o in orders:
        if o["created_at"]:
            try:

```

```

        dt = datetime.fromisoformat(o["created_at"])
        key = dt.strftime("%Y-%m")
        sales_by_month[key] += o["total"]
    except:
        pass

monthly_labels = sorted(sales_by_month.keys())
monthly_values = [sales_by_month[m] for m in monthly_labels]

# Commandes par jour
orders_by_day = defaultdict(int)
for o in orders:
    if o["created_at"]:
        try:
            dt = datetime.fromisoformat(o["created_at"])
            key = dt.strftime("%Y-%m-%d")
            orders_by_day[key] += 1
        except:
            pass

daily_labels = sorted(orders_by_day.keys())
daily_values = [orders_by_day[d] for d in daily_labels]

# 5 dernières commandes
def sort_key(o):
    try:
        return datetime.fromisoformat(o["created_at"])
    except:
        return o["id"]

recent_orders = sorted(orders, key=sort_key, reverse=True)[:5]

return render_template(
    "dashboard.html",
    total_customers=total_customers,
    total_products=total_products,
    total_orders=total_orders,
    total_revenue=total_revenue,
    sales_labels=sales_labels,
    sales_values=sales_values,
    stock_labels=stock_labels,
    stock_values=stock_values,
    top_products_labels=top_products_labels,
    top_products_values=top_products_values,
    monthly_labels=monthly_labels,
    monthly_values=monthly_values,
    daily_labels=daily_labels,
    daily_values=daily_values,
    recent_orders=recent_orders,
    best_product_name=best_product_name,
    best_product_qty=best_product_qty
)

# =====
# Lancement de l'application
# =====

if __name__ == "__main__":
    app.run(debug=True)

```

3. Notes importantes

✓ Clé secrète

Obligatoire pour :

- `flash()`
- `session`
- messages de validation
- formulaires

✓ Blueprints

Les modules suivants doivent exister :

- `customers/routes.py`
- `products/routes.py`
- `orders/routes.py`
- `categories/routes.py`
- `analytics/routes.py`

✓ Base de données

Le fichier `database/store.db` doit être créé via :

Code

```
python src/init_db.py
```

Mini Store Manager — `src/app.py` File

Author: Ezekiel Houab

Date: September 12, 2026

Project: Flask Application — Store Management

1. Introduction

This document presents the final and functional version of the `src/app.py` file used in the *Mini Store Manager* Flask application. This file:

- Initializes the Flask application.
- Configures the secret key for sessions.
- Registers the blueprints.
- Generates the dashboard.
- Runs the server in development mode.

This document is ready to be exported as a PDF.

2. Complete Code — src/app.py

```
from flask import Flask, render_template

from customers.routes import customers_bp

from products.routes import products_bp

from orders.routes import orders_bp

from categories.routes import categories_bp

from analytics.routes import analytics_bp

from database import (
    get_all_customers,
    get_all_products,
    get_all_orders
)

from collections import defaultdict

from datetime import datetime

app = Flask(__name__)

# =====
# SECRET KEY (required for flash(), session, etc.)
# =====

app.secret_key = "ezekiel_super_secret_key_2026"

# =====
# Blueprint Registration
# =====

app.register_blueprint(customers_bp)

app.register_blueprint(products_bp)

app.register_blueprint(orders_bp)

app.register_blueprint(categories_bp)

app.register_blueprint(analytics_bp)

# =====
# Dashboard
# =====
```

```

@app.route("/")

@app.route("/dashboard")

def dashboard():

    customers = get_all_customers()

    products = get_all_products()

    orders = get_all_orders()

    # KPIs

    total_customers = len(customers)

    total_products = len(products)

    total_orders = len(orders)

    total_revenue = sum(o["total"] for o in orders)

    # Sales by Product

    sales_by_product = defaultdict(float)

    qty_by_product = defaultdict(int)

    for o in orders:

        sales_by_product[o["product_name"]] += o["total"]

        qty_by_product[o["product_name"]] += o["quantity"]

    sales_labels = list(sales_by_product.keys())

    sales_values = list(sales_by_product.values())

    # Top 5 Products

    top_products = sorted(

        sales_by_product.items(),

        key=lambda x: x[1],

        reverse=True

   )[:5]

    top_products_labels = [name for name, total in top_products]

    top_products_values = [total for name, total in top_products]

    # Best-Selling Product (by Quantity)

    if qty_by_product:

        best_product_name, best_product_qty = max(

            qty_by_product.items(),

```

```

        key=lambda x: x[1]

    )

else:

    best_product_name, best_product_qty = None, 0

    # Stock by Product

    stock_labels = [p["name"] for p in products]

    stock_values = [p["stock"] for p in products]

    # Monthly Sales

    sales_by_month = defaultdict(float)

    for o in orders:

        if o["created_at"]:

            try:

                dt = datetime.fromisoformat(o["created_at"])

                key = dt.strftime("%Y-%m")

                sales_by_month[key] += o["total"]

            except:

                pass

    monthly_labels = sorted(sales_by_month.keys())

    monthly_values = [sales_by_month[m] for m in monthly_labels]

    # Orders by Day

    orders_by_day = defaultdict(int)

    for o in orders:

        if o["created_at"]:

            try:

                dt = datetime.fromisoformat(o["created_at"])

                key = dt.strftime("%Y-%m-%d")

                orders_by_day[key] += 1

            except:

                pass

    daily_labels = sorted(orders_by_day.keys())

```

```

daily_values = [orders_by_day[d] for d in daily_labels]

# 5 Most Recent Orders

def sort_key(o):
    try:
        return datetime.fromisoformat(o["created_at"])
    except:
        return o["id"]

recent_orders = sorted(orders, key=sort_key, reverse=True)[:5]

return render_template(
    "dashboard.html",
    total_customers=total_customers,
    total_products=total_products,
    total_orders=total_orders,
    total_revenue=total_revenue,
    sales_labels=sales_labels,
    sales_values=sales_values,
    stock_labels=stock_labels,
    stock_values=stock_values,
    top_products_labels=top_products_labels,
    top_products_values=top_products_values,
    monthly_labels=monthly_labels,
    monthly_values=monthly_values,
    daily_labels=daily_labels,
    daily_values=daily_values,
    recent_orders=recent_orders,
    best_product_name=best_product_name,
    best_product_qty=best_product_qty
)

# =====
# Application Launch
# =====

if __name__ == "__main__":

```

```
app.run(debug=True)
```

3. Important Notes

✓ Secret Key

Required for:

- `flash()`
- `session`
- Validation messages
- Forms

✓ Blueprints

The following modules must exist:

- `customers/routes.py`
- `products/routes.py`
- `orders/routes.py`
- `categories/routes.py`
- `analytics/routes.py`

✓ Database

The `database/store.db` file must be created using:

```
python src/init_db.py
```

✓ Image Illustration

The screenshot displays a web application interface for a mini-store manager. The dashboard features several key sections:

- Dashboard Overview:** Includes buttons for 'Produits', 'Clients', 'Commandes', and 'Revenu total' (displaying a Euro symbol).
- Produit le plus vendu:** A section indicating 'Aucune vente enregistrée' (No sales recorded).
- 5 dernières commandes:** A table listing the last five orders with columns for ID, Client, Date, Statut, and Total.
- Navigation Menu:** Located on the left, it includes links for Dashboard, Produits, Clients, Commandes, Catégories, Analytics, and Déconnexion.

The browser's developer tools are open, showing a warning message in the console:

```

Warning: Can't find variable: Category
    at http://127.0.0.1:5000/products/
  
```

The 'Produits' section is currently active, showing a table of products with columns for Nom, Catégorie, Prix, and Stock. Each product entry includes 'Modifier' and 'Supprimer' buttons.

←

127.0.0.1:5000/customers/

☆

Conversation

Dashboard

Produits

Clients

Commandes

Catégories

Analytics

Déconnexion

Clients

Rechercher un client...

Rechercher

Nom complet	Email	Ville	Commandes	Total dépensé	
	karim.belaid@example.com	Marseille		153.0 €	Voir détails
	emma.bernard@example.com	Nice		87.0 €	Voir détails
	bob.durand@example.com	Paris		147.8 €	Voir détails
	julien.fabre@example.com	Nice		241.0 €	Voir détails
	laura.gomez@example.com	Toulouse		810.9 €	Voir détails
	francois.lambert@example.com	Toulouse		188.0 €	Voir détails
	hugo.leclerc@example.com	Paris		67.2 €	Voir détails
	alice.martin@example.com	Lille		992.5 €	Voir détails
	david.morel@example.com	Marseille		114.0 €	Voir détails
	claire.petit@example.com	Lyon		123.4 €	Voir détails
	ines.robert@example.com	Lyon		774.6 €	Voir détails
	gina.rossi@example.com	Lille		151.0 €	Voir détails

←

127.0.0.1:5000/customers/

☆

Conversation

Dashboard

Produits

Clients

Commandes

Catégories

Analytics

Déconnexion

Clients

Rechercher un client...

Rechercher

Nom complet	Email	Ville	Commandes	Total dépensé	
	karim.belaid@example.com	Marseille		153.0 €	Voir détails
	emma.bernard@example.com	Nice		87.0 €	Voir détails
	bob.durand@example.com	Paris		147.8 €	Voir détails
	julien.fabre@example.com	Nice		241.0 €	Voir détails
	laura.gomez@example.com	Toulouse		810.9 €	Voir détails
	francois.lambert@example.com	Toulouse		188.0 €	Voir détails
	hugo.leclerc@example.com	Paris		67.2 €	Voir détails
	alice.martin@example.com	Lille		992.5 €	Voir détails
	david.morel@example.com	Marseille		114.0 €	Voir détails
	claire.petit@example.com	Lyon		123.4 €	Voir détails
	ines.robert@example.com	Lyon		774.6 €	Voir détails
	gina.rossi@example.com	Lille		151.0 €	Voir détails

←

127.0.0.1:5000/orders/

📄

☆

🔍

🌐

🗨️ Conversation

Dashboard

Produits

Clients

Commandes

Catégories

Analytics

Déconnexion

📄

Commandes

+ Créer une commande

ID	Client	Date	Statut	Total	
25	Alice Martin	2026-07-25	Payée	87.0 €	Voir détails
24	Laura Gomez	2026-07-24	Payée	22.9 €	Voir détails
23	Karim Belaid	2026-07-23	Livrée	34.0 €	Voir détails
22	Julien Fabre	2026-07-22	En cours	64.0 €	Voir détails
21	Inès Robert	2026-07-21	Payée	757.0 €	Voir détails
20	Hugo Leclerc	2026-07-20	Livrée	15.200000000000001 €	Voir détails
19	Gina Rossi	2026-07-19	Payée	40.0 €	Voir détails
18	François Lambert	2026-07-18	En cours	111.0 €	Voir détails
17	Emma Bernard	2026-07-17	Livrée	37.0 €	Voir détails
16	David Morel	2026-07-16	Payée	74.0 €	Voir détails
15	Claire Petit	2026-07-15	En cours	20.4 €	Voir détails
14	Bob Durand	2026-07-14	Livrée	43.0 €	Voir détails
13	Alice Martin	2026-07-13	Payée	199.0 €	Voir détails
12	Laura Gomez	2026-07-12	Payée	788.0 €	Voir détails
11	Karim Belaid	2026-07-11	Livrée	119.0 €	Voir détails
10	Julien Fabre	2026-07-10	En cours	177.0 €	Voir détails

←

127.0.0.1:5000/orders/

📄

☆

🔍

🌐

🗨️ Conversation

←

127.0.0.1:5000/orders/

📄

☆

🔍

🌐

🗨️ Conversation

Dashboard

Produits

Clients

Commandes

Catégories

Analytics

Déconnexion

📄

Commandes

+ Créer une commande

ID	Client	Date	Statut	Total	
25	Alice Martin	2026-07-25	Payée	87.0 €	Voir détails
24	Laura Gomez	2026-07-24	Payée	22.9 €	Voir détails
23	Karim Belaid	2026-07-23	Livrée	34.0 €	Voir détails
22	Julien Fabre	2026-07-22	En cours	64.0 €	Voir détails
21	Inès Robert	2026-07-21	Payée	757.0 €	Voir détails
20	Hugo Leclerc	2026-07-20	Livrée	15.200000000000001 €	Voir détails
19	Gina Rossi	2026-07-19	Payée	40.0 €	Voir détails
18	François Lambert	2026-07-18	En cours	111.0 €	Voir détails
17	Emma Bernard	2026-07-17	Livrée	37.0 €	Voir détails
16	David Morel	2026-07-16	Payée	74.0 €	Voir détails
15	Claire Petit	2026-07-15	En cours	20.4 €	Voir détails
14	Bob Durand	2026-07-14	Livrée	43.0 €	Voir détails
13	Alice Martin	2026-07-13	Payée	199.0 €	Voir détails
12	Laura Gomez	2026-07-12	Payée	788.0 €	Voir détails
11	Karim Belaid	2026-07-11	Livrée	119.0 €	Voir détails
10	Julien Fabre	2026-07-10	En cours	177.0 €	Voir détails

←

127.0.0.1:5000/orders/

📄

☆

🔍

🌐

🗨️ Conversation

←

127.0.0.1:5000/orders/

📄

☆

🔄

🔍

👤

⋮

🗨️ Conversation

Dashboard

Produits

Clients

Commandes

Catégories

Analytics

Déconnexion

📄

Commandes

+ Créer une commande

ID	Cliant	Date	Statut	Total	
25	Alice Martin	2026-07-25	Payée	87.0 €	Voir détails
24	Laura Gomez	2026-07-24	Payée	22.9 €	Voir détails
23	Karim Belaïd	2026-07-23	Livrée	34.0 €	Voir détails
22	Julien Fabre	2026-07-22	En cours	64.0 €	Voir détails
21	Inès Robert	2026-07-21	Payée	757.0 €	Voir détails
20	Hugo Leclerc	2026-07-20	Livrée	15.200000000000001 €	Voir détails
19	Gina Rossi	2026-07-19	Payée	40.0 €	Voir détails
18	François Lambert	2026-07-18	En cours	111.0 €	Voir détails
17	Emma Bernard	2026-07-17	Livrée	37.0 €	Voir détails
16	David Morel	2026-07-16	Payée	74.0 €	Voir détails
15	Claire Petit	2026-07-15	En cours	20.4 €	Voir détails
14	Bob Durand	2026-07-14	Livrée	43.0 €	Voir détails
13	Alice Martin	2026-07-13	Payée	199.0 €	Voir détails
12	Laura Gomez	2026-07-12	Payée	788.0 €	Voir détails
11	Karim Belaïd	2026-07-11	Livrée	119.0 €	Voir détails
10	Julien Fabre	2026-07-10	En cours	177.0 €	Voir détails